

Week 7: Planning, Exploration, and Advanced RL

Seasons of Code — Reinforcement Learning

Dyna-Q: Integrating Planning and Learning

Model-free methods (Q-learning, SARSA) learn purely from real environment transitions, which is sample-inefficient since each transition is used once and discarded. Model-based methods instead learn a model of the environment and plan with it, but a learned model is imperfect, so pure planning can be misled by model errors. **Dyna-Q** (Sutton, 1990) combines both: it learns from real experience *and* uses that experience to learn a model, which is then used to generate additional simulated updates.

Dyna-Q maintains three components:

- An action-value function $Q(s, a)$, updated by real and simulated transitions alike.
- A model $\text{Model}(s, a) \rightarrow (r, s')$, learned by simply recording the outcome of every real transition (deterministic table-lookup model in the tabular case).
- A planning loop that repeatedly samples previously-visited (s, a) pairs and updates Q using the model's predicted outcome, as if it were real experience.

Algorithm 1 Dyna-Q

```
1: Initialize  $Q(s, a)$  and  $\text{Model}(s, a)$  for all  $s, a$ 
2: for each real step do
3:    $s \leftarrow$  current state; choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
4:   Take action  $a$ , observe reward  $r$  and next state  $s'$ 
5:    $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$  ▷ direct RL update
6:    $\text{Model}(s, a) \leftarrow (r, s')$  ▷ model learning
7:   for  $n$  planning steps do
8:     Sample a previously observed  $(s, a)$ 
9:      $(r, s') \leftarrow \text{Model}(s, a)$ 
10:     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$  ▷ simulated RL update
11:   end for
12: end for
```

The parameter n controls the planning/learning tradeoff: $n = 0$ reduces Dyna-Q to plain Q-learning, while larger n extracts more value from each real transition by replaying it through the model many times, dramatically speeding up convergence in environments where real interaction is expensive. The cost is that the agent is only as good as its model; in stochastic or non-stationary environments, an inaccurate model can propagate biased value updates faster than real experience can correct them.

Planning vs. Learning Tradeoff

Planning uses a model to simulate experience and improve the value function or policy without further interaction with the real environment; it is cheap once a model exists but requires the model to be reasonably accurate. Learning uses real interaction, which is grounded in ground truth but is expensive/slow to collect and cannot be reused for a different purpose after being consumed. Dyna-style architectures make this tradeoff continuous rather than binary: the more accurate the model, the more planning steps are worth taking per real step.

Exploration: ϵ -Greedy vs. Softmax vs. UCB

ϵ -greedy. With probability $1 - \epsilon$ take the greedy action $\arg \max_a Q(s, a)$; with probability ϵ take a uniformly random action. Simple and guarantees infinite exploration of every action, but explores *undirected*: all non-greedy actions are equally likely regardless of how promising or well-understood they are, so it wastes samples on clearly bad actions just as often as on uncertain ones.

Softmax (Boltzmann) exploration. Actions are sampled proportional to $\exp(Q(s, a)/\tau)$. This is directed towards higher-value actions, but two actions with similar estimated value get similar selection probability even if one has been tried far more often (and is therefore known with more confidence) than the other — it does not explicitly account for *uncertainty*.

Upper Confidence Bound (UCB). UCB selects the action maximizing an optimistic upper bound on its value:

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right], \quad (1)$$

where $N_t(a)$ is the number of times a has been selected so far and c controls the exploration strength. The bonus term is large when an action has been tried rarely (high uncertainty) and shrinks as $N_t(a)$ grows, following the principle of *optimism in the face of uncertainty*: try what could plausibly be the best action, not just what looks best right now.

Comparison. ϵ -greedy explores blindly and its regret grows linearly with time in the worst case, since a constant fraction ϵ of actions remain wasted forever (this can be fixed with a decaying ϵ_t , but the decay schedule needs tuning). UCB explores in a directed, uncertainty-aware manner and achieves logarithmic regret $O(\log T)$ in the stochastic multi-armed bandit setting, which is the theoretically optimal rate. The price is that UCB assumes a well-specified, typically stationary reward distribution and requires visitation counts, which becomes harder to define and maintain once function approximation or continuous state spaces are involved — this is why ϵ -greedy (or its variants) remains the default in deep RL despite being less sample-efficient in principle, while UCB-style bonuses reappear in deep RL as intrinsic-motivation / count-based exploration bonuses rather than in their original tabular bandit form.

Exploration Challenges in Deep RL

Classical exploration methods rely on exact state/action visitation counts, which do not generalize to large or continuous state spaces where nearly every state is visited at most once. Deep RL exploration must instead use function approximation to *generalize* notions of novelty or uncertainty across similar states (e.g. pseudo-counts, prediction-error-based curiosity, or posterior sampling over an ensemble of value functions), and must do so while the value/policy network is simultaneously

changing, which the classical bandit analysis does not account for. Sparse-reward, long-horizon tasks make this substantially harder, since undirected exploration (ϵ -greedy) may never stumble onto a reward signal at all.

Limitations of Classical RL and the Move to Deep RL

Classical (tabular) RL requires a lookup table over the full state(-action) space, which becomes infeasible as state/action dimensionality grows and provides no way to generalize value estimates between similar-but-unvisited states. Function approximation (Week 5) addresses this by parameterizing Q or π , allowing generalization across a continuous or high-dimensional state space, at the cost of losing tabular convergence guarantees and introducing the instability discussed with DQN (deadly triad). Deep RL scales this further using neural networks as the approximator, enabling RL to operate directly on raw, high-dimensional observations (pixels, sensor arrays) instead of hand-engineered features, which is what makes it applicable to complex environments like robotics and vision-based control that classical RL cannot practically address.

Actor-Critic Methods: PPO, DDPG, SAC

Actor-critic methods maintain two components: an **actor** $\pi_\theta(a \mid s)$ that selects actions, and a **critic** $Q_w(s, a)$ or $V_w(s)$ that evaluates them, replacing the raw Monte Carlo return G_t used in REINFORCE (Week 6) with a learned, lower-variance estimate of value. This resolves the actor's need for full-episode rollouts before it can update.

Algorithm	Strengths	Weaknesses
PPO	On-policy, clipped surrogate objective keeps updates within a trust region without the complexity of exact second-order methods (TRPO); simple to implement and tune; stable across a wide range of tasks; works for both discrete and continuous action spaces.	On-policy: cannot reuse old roll-outs once the policy has changed, making it sample-inefficient compared to off-policy methods; still needs a reasonably large batch of fresh trajectories per update.
DDPG	Off-policy with a replay buffer, so it reuses past experience and is more sample-efficient than PPO; deterministic actor makes it natural for continuous, high-dimensional action spaces (e.g. robotic control).	Deterministic policy means little inherent exploration (needs external action noise, e.g. Ornstein-Uhlenbeck); notoriously sensitive to hyperparameters and prone to instability/divergence; brittle, often requires extensive tuning per task.
SAC	Off-policy and sample-efficient like DDPG, but adds an entropy-maximization term to the objective, which explicitly rewards exploration and yields a stochastic policy; empirically much more stable and robust to hyperparameters than DDPG; strong performance on continuous control benchmarks.	More complex to implement (twin critics, entropy temperature α tuning); the entropy coefficient still requires care (though auto-tuning variants mitigate this); higher computational cost per update than PPO due to the twin critic and entropy machinery.

Summary. PPO trades sample efficiency for simplicity and stability and is the default choice when reliable, easy-to-tune training matters more than sample count. DDPG demonstrated that off-policy actor-critic methods could handle continuous control efficiently, but its determinism and instability limit practical use. SAC largely supersedes DDPG for continuous control by keeping its sample efficiency while fixing its stability and exploration problems through entropy regularization, at the cost of extra implementation complexity.